

Rule Engine Research and Implementation in Financial System

Hongjin ZHANG, Xiaohu YANG
 College of Computer Science and Technology
 Zhejiang University
 Hangzhou, China
chariot@zju.edu.cn, yangxh@zju.edu.cn

Abstract—With the quick development of economy, the market changes so rapidly that the business logics are forced to change frequently. In an original real-time financial system, the cost of maintenance was very high since the business logics were mixed up with the code at application level. This paper elaborated a kind of rule engine implementation mode for the financial system. The rule engine separated the business logics from the application program. This strategy can make the system flexibility to the rapid change very well without increasing the cost.

Keywords—Financial System; Business Rules; Rule Engine; Rete

I. INTRODUCTION

With the quick development of economy, the market changes so rapidly that the business logics are forced to change frequently. This requires IT companies to design software systems that can adapt to the quick change very well in order to reduce the cost of development and maintenance. So there comes the Rule-based Expert System [8]. Rule engine [7], which is an embedded component in the system, can achieve this aim by separating the business logics from the application program [3]. The business rules are written by a pre-defined language. The rule engine accepts the data inputs, explains the business rules and then makes the business decisions. When the business logics change, the rules change, and other parts of the system including the designs and codes remain intact. The business rules can be maintained independently according to the business requirements. There are some popular rule engines and Business Processing Manager system in the present market (BRM [1, 6], they implement some rule engine algorithms such as Rete [2,4], LEAPS[5] etc).

Nowadays, the large financial systems cover many kinds of business and trade logics, and these logics change very frequently. In the original financial system, the business logic codes were mixed up with the application layer program, so the maintenance cost was very high. We need a rule engine to reduce the cost and make the system more flexibility effectively.

This paper described a rule engine implementation for the financial system. It maintains the business rules using the self-defined rule language (*.rb files), translates the business rules to Java Class binary files

and then saves them into database. The application layer can execute these rules using the rule engine API. This rule engine also provides a configuration manager tool; the client can customize the business rules as their requirement individually and interference-freely.

II. ARCHITECTURE OVERVIEW

There are lots of advantages to using Rule Engine in enterprise information system. Business rules significantly reduce the cost of requirement analysis, coding, integration testing and deployment. The development cycle becomes smaller to adapt the rapid change of business logics. The developer can just focus on the business rules by ignoring the programming technique, since the business rules can be close to the natural language. So they can be more easily comprehended and maintained.

Considering this financial system feature, we designed a Financial Rules Manager for this system (short for FRM). FRM is interacting with application work flow component closely; it is offering the functions of business trade information validation, enrichment and translation. The Figure 1 showed the architecture of FRM.

The core of FRM includes Rule Base Manager, Rule Configurator, and Rule Processor; the Reference Data System provides some outside trade data for processing.

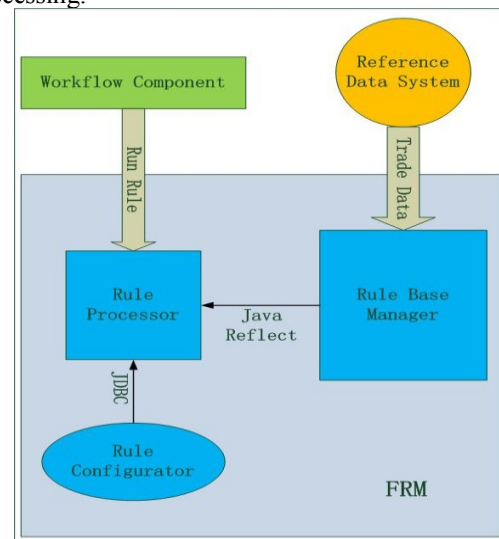


Figure 1. FRM Architecture

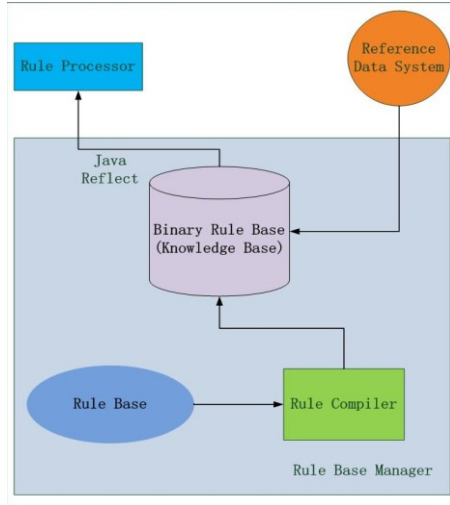


Figure 2. Rule Base Manager

The whole business rules processing is independent of the workflow component. The workflow component calls the API of FRM, FRM queries out the rules sets for the assigned client, after running these rules sets, it returns the result to the workflow component.

III. RULE BASE MANAGER

Rule Base Manager administers the financial business rules, interacts with Rule Processor and Reference Data System. It contains Rule Base, Rule Compiler and Binary executable Rule Base (we can call it Knowledge Base). Rule compiler translates the business rules to binary Java class files and saves them into database to constitute the Knowledge Base, and Rule Processor can call the executable rules in the Knowledge Base through Java reflection mechanism. See the Rule Base Manager architecture in Figure 2.

A. Rule Base

The business rules are saved in plain text format, organized by the different activities in financial processing. The grammar for this business rule is very simple. Each rule just has two parts - predicate and action, using IF and THEN-ELSE key words to express. One rule can call other rules in processing to achieve the rule reusing. The most basic rules just contain the predication for one or more context attributes and the action, like BASIC_Rule1_EnrichAttr3, called Basic Rule. It is the smallest also the most common unit in the logic rule reusing. Please see the following sample rules set:

```
OBJECT TradeObject IRules {
    double attr1;
    double attr2;
    double attr3;
    String errorCode;
    void setError();
}
RULE BASIC_Rule1_EnrichAttr3 {
```

```
    VALUEPROPERTY vp;
    IF (TradeObject.attr1 > 0 &&
        TradeObject.attr2 > 0 )
    THEN {
        TradeObject.attr3 = TradeObject.attr1 +
        TradeObject.attr2;
    }
}
RULE BASIC_Rule2_ValidateAttr3 {
    VALUEPROPERTY vp;
    IF (TradeObject.attr3 > 0 )
    THEN {}
    ELSE {
        TradeObject.setError();
        TradeObject.errorCode = "ErrorCode1";
    }
}
RULECP RuleCP_EnrichAttr3 {
    VALUEPROPERTY vp;
    IF (BASIC_Rule1_EnrichAttr3.vp==true &&
        BASIC_Rule2_ValidateAttr3.vp == false)
    THEN {}
}
RULECP FR_EnrichAttribute3 {
    VALUEPROPERTY vp;
    IF (RuleCP_EnrichAttr3.vp == true)
    THEN {}
}
```

It is necessary to use key word 'OBJECT' to declare all the variables in the beginning, OBJECT means the context information for the financial trade. TradeObject is a hash map essentially, it implements the IRules interface (this interface defines all the basic methods for running the rules). In TradeObject, the attr1, attr2 are the attribute objects corresponding to the trade information.

The definition for each rule starts with the key word RULE or RULECP. The rule name follows them. RULE represents the Basic rule, RULECP means compound rule, which contains one or more Basic rule in its body. The rules named 'FR' at the beginning are the fire rules (or executable rules), they are the smallest running and configuration units for Rule Configurator and Processor.

VALUEPROPERTY vp denotes the return value for the rule. The expression after the key word IF is the rules predicate, it supports the any combination of logic operators. If the result of IF expression is true, vp is assigned true value implicitly, the actions in THEN are fired, otherwise, vp is assigned false value implicitly, and the actions in ELSE are fired. If there is nothing to do in ELSE, it can be omitted.

B. Rule Compiler

Rule Compiler turns the text rules into binary executable rules. There are three steps while compiling. Firstly, it translates the text rules into Java file temporarily. Then it calls Java compiler to compile this temporary Java file to generate the binary Java class.

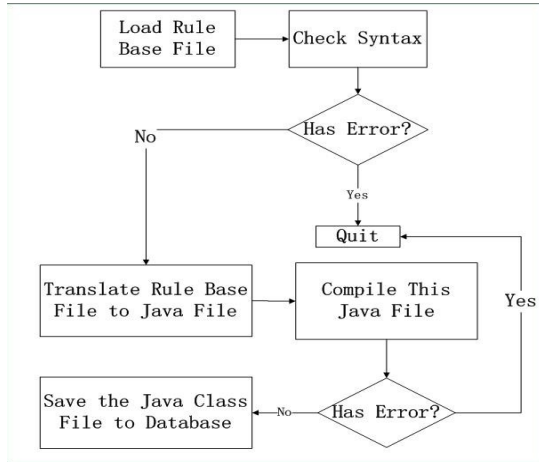


Figure 3. Rule Base Compiler

At last, it saves the binary Java class into database. The figure 3 demonstrates the Rule Compiler processing step.

C. Binary Rule Base

After the Rule Compiler working, the business rules has been turned into binary rules and saved into data base, so the Binary Rule Base is created. The Rule Processor can call the binary rules through the Java reflection mechanism (this will be described in Rule Processor chapter).

D. Reference Data System

In rule processing, it needs to access the Reference Data System (RDS) to get the trade information for validation, enrichment and translation. FRM uses CORBA for exchanging data with Reference Data System. FRM defines an abstract class *DataAccess*, which offers some methods to access the interfaces of RDS. The subclasses of *DataAccess* define the concrete methods for their special requirements. But in rule processing, rules do not call these *DataAccess* classes to access the RDS. Although the rule grammar allows calling outside Java method, it would make the rules tight coupling with outside component. So FRM defines a *ReferenceDataObject* class, which implements the *IRules* interface. This object will be imported as a parameter with context information when running the rules. While the rules need to access the RDS, they use *ReferenceDataObject* in place of calling RDS interface directly.

IV. RULE CONFIGURATOR

Financial system covers many kinds of the business, different client has distinct requirement. We need a good management approach for business rules and client information to make the system flexible and easily expanded. So we design a Rule Configurator to achieve this purpose.

Rule Name	Rule ID	Enabled
FR_EnrichAttribute1	000001	☒
FR_EnrichAttribute2	000002	☐
FR_EnrichAttribute3	000003	☒
FR_CheckAttribute1	000004	☒
FR_CheckAttribute2	000005	☐
FR_CheckAttribute3	000006	☒
FR_ValidateSettlement	000007	☒
FR_TranslateAttribute4	000008	☐

Figure 4. Rule Configuration

According their requirements, clients can chose the rule processing set in activities or change the running sequence, without affecting the other clients. The rule configuration can be changed dynamically, the system does not need to reboot.

Figure 4 illustrates the demo UI for rule configuration. Activity means the trade processing stage, in other words, we divide the whole trade processing into some stage, and each stage does the special function. Apparently, different client has distinct rule configurations, and we also support *CopyTo* function to copy one client configuration to another. Rule ID means the serial number, which is the primary key for the rule. If the Enabled check box is selected, this rule will be executed in the activity.

As mentioned earlier, there are some processing stages (activities) in the whole transaction, in each activity, the business rules may need to be run depending by the client. The activity contains the largest collection for the rules for each stage, and the client rule set is the subset of it. Figure 5 demonstrated the relation among rule, activity and client.

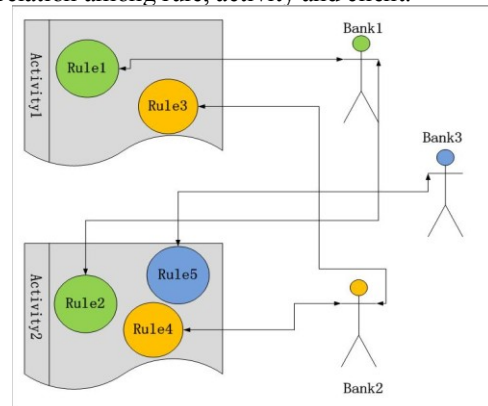


Figure 5. Rule – Activity – Client

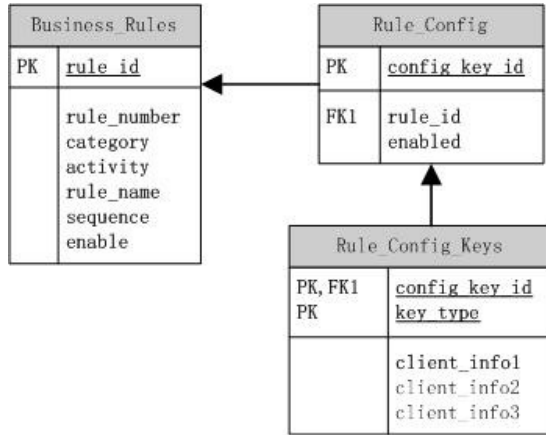


Figure 6. Rule Configurator Data Base Structure

We keep the configure information into database. According to the feature of the financial system, we designed the database for configuration as the above figure.

Table *Business_Rules* is used to store the basic information of rules, including *rule_id*, *rule_number*, *rule_name*, *activity*, *enable*, etc. Table *Rule_Config* is used to associate *Business_Rules* with client information. *Rule_Config_Keys*, of course, is used to keep the client information.

V. RULE PROCESSOR

Rule Processor (RP) offers an interface to workflow component, *ProcessRule.run* (*TradeObject*, *Activity*) to call the binary rule sets. The first parameter *TradeObject* is the context information including trade information and client information; the second parameter is activity info. There are five main processing steps:

- Load rule configuration information;
- Load the enabled binary rules (from cache or DB);
- Get *ReferenceDataObject*;
- Load context info to the rules;
- Fire the enabled rules in sequence;

RP queries out the enabled rules from the client info (*TradeObject* provides) and activity info and then loads these binary rules from data base or cache. RP uses Java reflection mechanism in place of directly calling. Java reflection mechanism allows program using Reflection APIs to get the internal information of a known class in runtime.

Before setting the context, RP has already get the Java class file which contains the enabled rules from

data base or cache, then RP saved this class into a session:

```
session = ruleClass.newInstance();
```

When setting the context, RP will read the method named 'setContext' from the binary rule Java class file. In order to read correctly, RP need offer the method name and parameters, so RP uses *context.getClass()* procedure to get the context type. See the following code segment for executing the method through Java reflection.

```
Class[] params = new Class [1];
params[0] = context.getClass ();
Object[] args = new Object [1];
args[0] = context;
String methodName = "setContext";
Method method;
try {
    method = ruleClass.getMethod(methodName, params);
    method.invoke(session, args);
}
```

After running the rules, the information of *TradeObject* has been validated, enriched and translated. RP then returns the value to the workflow component.

VI. CONCLUSION

In this paper, we introduced an implementation approach of rule engine for a financial system, called FRM. FRM implemented some basic functions for BRMS, such as separation of business logics, using simple language to design the business rules. FRM also can administrate the business rules dynamically.

REFERENCES

- [1] Xiaoming Feng, Mani Subramanian. "Incorporating Business Rule Engine Technology in Control Center Applications". IEEE Energy2030 Atlanta, GA USA 17-18 November, 2008.
- [2] Production Matching for Large Learning Systems. Robert B. Doorenbos. January 31, 1995. CMU-CS-95-113.
- [3] TAO Xiaojun, ZHU Min. "Pattern of Building Enterprise Services Based on Rule Engine". Computer Technology And Development, China. Vol.18, No.2, Feb. 2008.
- [4] Charles L. Forgy. "Rete: A fast algorithm for the many pattern/many object pattern match problem". Artificial Intelligence. 1982. Volume 19, Issue 1, Pages 17-37.
- [5] Don Batory. "The LEAPS Algorithms". Technical Report 94-28 of Department of Computer Sciences, University of Texas at Austin. 1994.
- [6] <http://en.wikipedia.org/wiki/BRMS>
- [7] Chisholm M. "How to Build a Business Rules Engine" Morgan Kaufmann, 2003
- [8] Peter Jackson. Introduction to Expert Systems. Addison-Wesley Publishing Company. 1990. 3rd Edition.